SOFTWARE ENGINEERING DOCUMENTATION

Level Up Primary Learning Path
Software Requirements Specification

Interactive Linear Learning System for Elementary Education

Lecturer:

Hendra Jayanto

Class:

Artificial Intelligence 1

Created by:

1. NAUFAL RIZKI PINUGROHO 001202400097
2. DARRELL RAFA ALAMSYAH 001202400018
3. MUHAMMAD DZAKI ABRAR 001202400181
4. MUHAMMAD DHIYA ULHAQ 001202400204
5. FATHI IHSAN PRIYAYI 001202400000
6. LIU JUNMIN 001202500238

INFORMATICS

FACULTY OF COMPUTER SCIENCES

PRESIDENT UNIVERSITY

2025/2

TABLE OF CONTENT

1. INTRODUCTION

1.1 Purpose

This Software Requirements Specification (SRS) document serves as a comprehensive formal agreement between the development team and the stakeholders of the **Level Up** platform. Its primary purpose is to delineate every functional capability and non-functional constraint of the system, providing a definitive "source of truth" for the implementation phase. By detailing the technical logic and user expectations, this document facilitates accurate project estimation, risk management, and quality

assurance testing throughout the software development life cycle (SDLC). Furthermore, it fulfills the academic requirements for the Software Engineering course under the supervision of Hendra Jayanto at the Faculty of Computer Sciences, President University.

1.2 Background

The evolution of digital education has led to a proliferation of e-learning platforms; however, many existing systems for elementary education function merely as digital repositories of static content. This "passive" learning model often fails to ensure that a student has truly mastered a topic before moving to more advanced material. MasteryPath was conceived to bridge this gap by integrating Linear Gamification and Mastery-Based Pedagogy.

Unlike traditional platforms where a student might skim through modules, Level Up utilizes a strict sequential unlocking mechanism. This ensures that the learning path is structured and logical, preventing the cognitive overload that occurs when students encounter advanced problems without a solid understanding of the prerequisites. By utilizing modern web technologies like Next.js 19 and Appwrite, the platform provides a responsive, real-time environment where academic progress is treated as an interactive journey, fostering a sense of accomplishment and continuous growth.

1.3 Problem Statement

The development of Level Up is driven by the need to solve four critical issues currently prevalent in retail e-learning solutions for children:

1. Passive Engagement and Information Retention: Most platforms lack interactive feedback loops. Without a sense of "gameplay" or active participation, students become passive consumers, leading to lower information retention and high bounce rates.
2. Unaddressed Knowledge Gaps: Standard systems often allow "free-roaming" through the curriculum. This lack of structure allows students to avoid difficult topics, creating "gaps" in their foundational knowledge that eventually hinder their ability to solve complex problems in higher levels.
3. Lack of Contextual Relevance: Traditional test questions are often abstract and disconnected from a child's daily interests. This disconnect makes core subjects like Math and Science feel like "chores" rather than applicable skills.
4. Interface Over-Complexity: Many platforms utilize corporate-style UX patterns that are visually intimidating and not optimized for the specific motor skills and shorter attention spans of elementary-aged users. There is a clear need for a "Glassmorphism" based, high-contrast, and intuitive UI.

> **Core Problem:** How can a web-based e-learning platform utilize linear gamification and adaptive interest-profiling to ensure mastery-based knowledge retention and high user engagement for elementary students, without the need for manual teacher intervention?

1.4 Project Objectives

To address the aforementioned challenges, the Level Up project aims to achieve the following specific, measurable objectives:

1. Establish a Linear Progression Engine: Build a 24-node sequential curriculum map where the backend strictly enforces access controls, ensuring Node N+1 is only accessible after the successful completion of Node N.
2. Implement an Adaptive Placement System: Design an automated Diagnostic Engine that evaluates a student's baseline upon registration, effectively placing them into Foundation, Beginner, or Intermediate tiers to match their current proficiency.
3. Develop Dynamic Interest-Based Logic: Create a profiling system (supporting Computer, Sport, and Art) that dynamically modifies the "flavor text" and context of exam questions, making the curriculum personally relevant to each student.
4. Ensure Real-Time Data Synchronization: Leverage Appwrite to maintain a persistent state of student achievements, including total Experience Points (EXP), scores, and exact map positioning, allowing for seamless cross-device learning sessions.
5. Optimize Performance and Aesthetics: Deliver a high-performance web interface using the latest Next.js 19 (App Router) and Tailwind CSS v4, utilizing micro-animations and "Glassmorphism" to create a visually stimulating yet professional environment.

1.5 Scope of the System

Included in Scope

- Authentication & User Profiling: Complete registration and login modules with secure session management via Appwrite.
- Onboarding & Diagnostic Testing: A mandatory initial workflow that assesses proficiency and captures student interest preferences.
- 24-Node Sequential Curriculum Map: A visual dashboard that renders progress and restricts unauthorized access to future modules.
- Interactive Learning & Exam Modules: Dedicated pages for content consumption and assessmet with real-time scoring logic.
- Scoring, Rewards, and Analytics: Tracking of EXP, score percentages, and time-to-complete metrics stored persistently in the cloud.
- Administrative Quality Assurance (QA) Tools: An "Admin Bypass" system allowing authorized users to bypass progression locks for content auditing and system testing.

Excluded from Scope

- Live Pedagogical Interaction: The system does not support real-time video conferencing or direct instant messaging with teachers.

- Native OS Mobile Applications: The project focuses on a "Web-First" responsive strategy; native Android or iOS builds are categorized as future work.
- External Database Integration: Synchronization with national education databases (like Dapodik) is currently outside the technical scope.

1.6 Target Users

The system is designed for three distinct user classes, each with specific requirements:

User Type	Profile	Primary Use
Student	Primary users (Ages 7–12) in elementary school.	Navigating the linear map, consuming modules, and passing exams to earn EXP and unlock new nodes.
Educator / Admin	Teachers or System Developers.	Utilizing the Admin Bypass for curriculum review, content auditing, and troubleshooting progression logic.
Researcher / Student	Academics or fellow Informatics students.	Analyzing the efficacy of linear gamification and BaaS (Appwrite) integration in educational software.

2. LITERATURE REVIEW

2.1 Mastery-Based Learning (MBL)

Mastery-Based Learning is an instructional strategy and educational philosophy first formally proposed by Benjamin Bloom in 1968. Unlike traditional models where students move through a curriculum based on a fixed schedule regardless of their understanding, MBL requires students to demonstrate a thorough grasp of a topic typically defined as a 80% to 90% score on a summative assessment before advancing.

In the context of MasteryPath, MBL is the core operational logic. By utilizing automated assessment engines, the system ensures that foundational concepts in Math, Science, and English are solidified before the student encounters complex modules. This approach addresses the "2 Sigma Problem," which suggests that students who learn via mastery-based tutoring perform significantly better than those in conventional classrooms. MasteryPath digitizes this "tutor" role through its automated exam and progression logic.

2.2 Pedagogical Scaffolding and Linear Gamification

Instructional "Scaffolding" refers to the support structures provided to students as they develop new skills. In software engineering, this is often implemented through Linear Gamification, a design pattern where content is partitioned into sequential "nodes" or "levels." This pattern prevents "cognitive overload," a state where a learner's working memory is overwhelmed by too much information at once.

MasteryPath utilizes a 24 node linear map as a visual scaffold. By locking future nodes, the system creates a "Flow State" (as defined by Mihaly Csikszentmihalyi), balancing the challenge of the material with the student's current skills. This gamified approach transforms the perception of educational rigor into a series of achievable "quests," significantly increasing user retention compared to non-linear e-learning platforms.

2.3 Next.js 19 and Server-Side Rendering (SSR) Architecture

Modern web applications, especially those serving data-sensitive educational content, require a balance between high-speed performance and secure data fetching. Next.js 19, utilizing the App Router architecture, provides a robust framework for this purpose.

1. Server-Side Rendering (SSR): Unlike traditional Client-Side Rendering (CSR), SSR generates the HTML for a page on the server before sending it to the user. This is critical for the MasteryPath "Map" dashboard; the system can check the student's progress in the database and render the locked/unlocked state of the 24 nodes before the page even loads in the browser, eliminating "flicker" and enhancing security.
2. Component-Based UI: React 19 allows the platform to utilize modular components, making the "Exam Engine" and "Learning Modules" highly maintainable and reusable.

2.4 Backend-as-a-Service (BaaS) and Data Persistence

For the Level Up platform, managing user authentication, proficiency levels, and real-time progress requires a flexible and scalable backend. Appwrite is utilized as the Backend-as-a-Service (BaaS) provider. Traditional relational databases (SQL) often require complex migrations for changing schemas. In contrast, Appwrite's document-based database allows for a "Dual Persistence Architecture". This ensures:

1. Cloud Synchronization: Every time a student earns EXP or passes a node, the data is instantly synced to the Appwrite cloud.

2. Session Management: Secure handling of user identities through the account and session APIs, ensuring that student data remains private and protected via HTTPS.

2.5 Personalization and Adaptive Interest Profiling

Research in educational psychology indicates that student engagement increases by nearly 30% when the context of the learning material aligns with the student's personal interests. This is known as "Personalized Contextualization."

Level Up implements this through Interest Profiling (Computer, Sport, Art). By tagging the question bank with interest metadata, the system's "Dynamic Exam Engine" can filter and retrieve questions that frame Math or Science problems within a context the child already finds exciting. This reduces "learning friction," making the assessment feel less like a test and more like a hobby-related challenge.

2.6 Related Systems

A comparative analysis of existing e-learning solutions highlights the unique niche occupied by Level Up:

Platform	Strengths	Gap vs Gold Analysis Platform
Khan Academy	Vast library of professional content.	Lacks interest-based question tailoring; navigation can be overwhelming for young children.
Duolingo	Excellent use of gamification and streaks.	Strictly focused on language; does not apply linear mastery to general subjects like Math/Science.
Google Classroom	Strong teacher-student management.	Primarily a repository; does not feature an automated, gamified linear map for independent study.
Level Up	**100% Linear, Free, and Interest-Adaptive.**	**Combines a 24-node map with interest profiling to ensure mastery in core subjects.**

3. FUNCTIONAL REQUIREMENTS

The functional requirements (FR) for Level Up - Primary Learning Path define the specific behaviors, services, and operations the system must perform to fulfill the learning objectives of the student and the administrative needs of the educator. These requirements are categorized by priority: Critical (essential for the system to function), High (vital for the core user experience), and Medium/Low (enhancement features).

ID	Feature	Detailed Description	Acceptance Criteria	Priority
FR-01	Global Authenticatio n & Session Management	The system must allow users to create accounts and log in using an email and password via the Appwrite Authentication service. Sessions must remain persistent to prevent data loss during accidental browser refreshes.	1. User can register and login. 2. Session persists across page reloads. 3. Secure logout clears all local and server-side tokens.	Critical
FR-02	Automated Diagnostic Placement Engine	Upon the initial login, the system shall redirect the student to a 10-question diagnostic exam covering Math, Science, and English. The system calculates a proficiency score to categorize the user.	1. Redirect occurs only for new users. 2. System assigns level: Foundation, Beginner, or Intermediate. 3. Result is saved to the Appwrite profiles collection.	Critical
FR-03	Interactive Interest Profiling	The system shall prompt the user to select one of three interest categories: Computer, Sport, or Art. This selection is used as a filter for the Dynamic Exam Engine.	1. User selection is saved to the cloud. 2. Selection can be viewed in the profile menu. 3. Changing the selection updates future exam questions.	High
FR-04	Linear Dashboard & 24-Node Map	The main dashboard must render a visual path consisting of 24 distinct nodes spread across a multi-floor layout. This serves as the primary navigation hub for the curriculum.	1. Exactly 24 nodes are rendered.	Critical

			2. Nodes reflect the current progress state (locked/unlocked). 3. Layout is responsive for tablet and desktop users.	
FR-05	State-Driven Progression Lock	The system must strictly enforce a linear path. If a student is currently at Node N, all nodes from $N+1$ to 24 must be unclickable and visually represented with a locking indicator.	1. Attempting to click Node N+1 results in no action or a "locked" alert. 2. Padlock icons are visible on all future nodes. 3. Completed nodes remain accessible for review.	Critical
FR-06	Learning Module Content Interface	Clicking an unlocked node must navigate the user to a dedicated page containing the educational materials (text and visuals) relevant to that specific node's topic.	1. Content loads based on the nodeID. 2. "Start Exam" button is only visible after the user lands on this page. 3. Material is optimized for readability.	High
FR-07	Dynamic Contextual Exam Engine	The system shall load a quiz where the questions are filtered by both the Node topic and the Student's chosen interest. For example, a Math question for a "Sport" user might involve calculating athlete statistics.	1. Question bank is filtered by interest_tag. 2. System tracks answers without data loss. 3. Score calculation occurs immediately upon submission.	High

FR-08	Progress Synchronization & Rewards	If a student passes an exam, the system must update the currentNode attribute in the Appwrite database and award Experience Points (EXP).	1. Database is updated via databases.updateDocument. 2. Results Modal displays Score, Time, and total EXP earned. 3. Next node is automatically unlocked upon return to the map.	High
FR-09	Dual-Theme Customization Engine	The system shall provide a theme toggle in the user settings. Switching to the "High Contrast" or "Alternative" theme must swap CSS variables and background assets instantly. .	1. Theme selection is saved to the Appwrite profile. 2. Visual changes apply across all dashboard components. 3. Theme remains active across subsequent logins.	Medium
FR-10	Administrative Bypass Protocol	The system must recognize a pre-defined Administrative User ID (e.g., student1). This role bypasses the isLocked logic for all 24 nodes to facilitate system auditing.	1. Logic checks User ID during the dashboard mount. 2. All 24 nodes are set to unlocked: true for the Admin. 3. Admin can enter any module at any time.	Medium

4. NON-FUNCTIONAL REQUIREMENTS

While functional requirements define *what* the system does, non-functional requirements (NFR) define *how* the system performs. For a platform targeting elementary students, factors such as response time, visual accessibility, and data integrity are critical to maintaining engagement and trust.

ID	Category	Requirement	Target
NFR-01	Performance	Initial Dashboard Load Time: The time taken for the Next.js server to render the 24-node map upon authentication.	< 2.5 seconds under standard broadband conditions.
NFR-02	Performance	State Transition Latency: The delay when navigating from a Learning Module to the Exam Engine.	< 500ms to maintain student focus levels.
NFR-03	Usability	Interface Accessibility: The UI must adhere to "Child-Friendly" design principles, utilizing large touch targets and high-contrast text.	Use of Glassmorphism (backdrop-blur) and minimum 18px font size for core content.
NFR-04	Usability	Mobile Responsiveness: The platform must adapt its 24-node layout to various screen orientations.	Full compatibility with screens >= 360px (mobile) and >= 768px (tablets).
NFR-05	Security	Data Encryption: All data transmitted between the React client and Appwrite cloud must be encrypted.	Enforcement of HTTPS/TLS 1.2 or higher for all API calls.
NFR-06	Security	Access Control: Prevention of "Direct URL Hacking" where a student attempts to access Node $N+5$ manually.	Middleware-level redirection to the last unlocked node index.
NFR-07	Availability	System Uptime: The platform should be accessible for students to study at any time.	99.9% uptime (leveraging Vercel and Appwrite SLA).
NFR-08	Maintainability	Code Quality: The system must be built using modular architecture to allow for future curriculum expansion.	100% TypeScript strict mode compliance and component reusability.

4.1 User Requirements

The following requirements represent the expectations of the end-users to ensure the platform provides a meaningful educational experience:

- Engagement and Feedback: Students require immediate visual feedback (e.g., animations or sound cues) upon passing an exam to reinforce the sense of achievement.

- Simplified Navigation: Young users require a "Zero-Instruction" UI where the next step is always visually prioritized (e.g., a glowing effect on the currently unlocked node).
- Persistent Progress: Users expect to close their browser on a tablet and resume exactly where they left off on a desktop computer without losing EXP or progress.
- Multilingual Support: While the core curriculum is in English, the UI elements should be intuitive enough that students with varying English proficiency can navigate the system.
- Transparency: Students should be able to see *why* they received a certain score, with a clear summary of correct vs. incorrect answers provided in the Results Modal.

4.2 System Constraints

The development and operation of Level Up - Primary Learning Path are subject to several technical and environmental limitations:

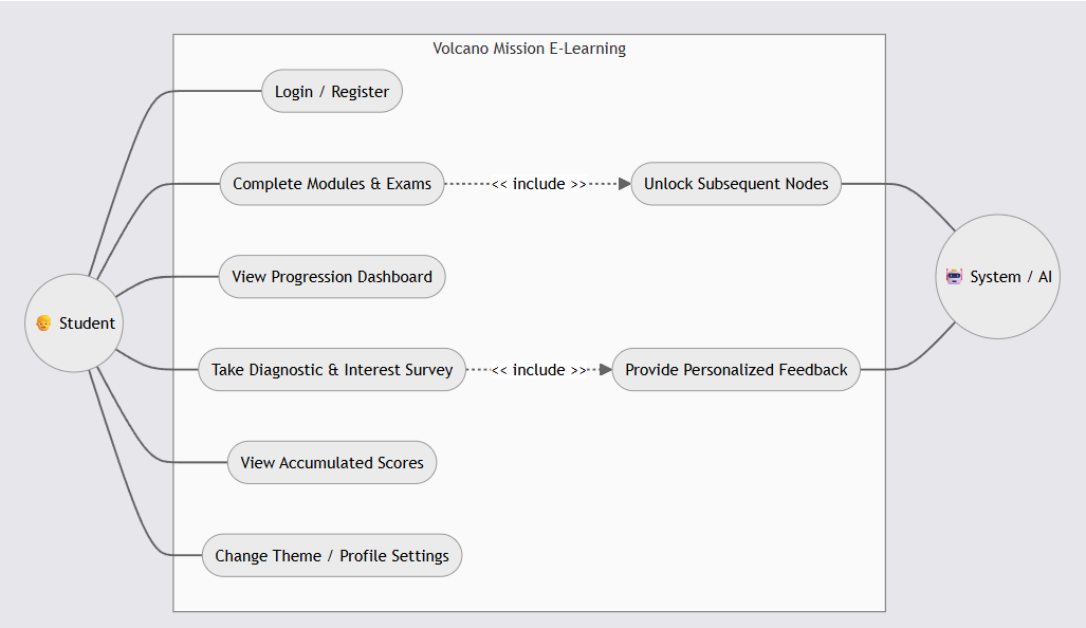
- Connectivity Dependency: The system requires an active internet connection to synchronize progress with the Appwrite database; offline mode is currently not supported.
- BaaS Rate Limits: The platform is subject to the request-per-minute limits of the Appwrite free tier, which may impact performance if more than 50 students access the system simultaneously.
- Browser Compatibility: The application is optimized for modern evergreen browsers (Chrome, Edge, Safari, Firefox). Older versions of Internet Explorer are strictly unsupported due to the use of Tailwind CSS v4 and Next.js 19 features.
- Hardware Limitations: Low-end mobile devices with less than 2GB of RAM may experience "jank" during the rendering of complex SVG assets and backdrop-blur effects.
- Academic Compliance: The system is developed as a final project for President University and must adhere to the specific grading rubrics and ethical guidelines set by the Faculty of Computer Sciences.

5. USE CASE DESCRIPTION

5.1 Use Case Overview

The use case diagram serves as a functional blueprint for the Level Up - Primary Learning Path platform, illustrating the dynamic interactions between the users and the system's automated logic. This visualization ensures that all features align with the primary goal of providing a structured, mastery-based educational journey through gamified progression.

5.1.1 Use Case Diagram



5.1.2 Actor Descriptions

The system architecture defines three distinct actors to facilitate the learning lifecycle:

- Student (Primary Actor): The central user who engages with the curriculum, navigates the map, and completes assessments to progress through the 24 learning nodes.
- System / AI (Secondary Actor): Represented in the diagram as the automated backend layer. This actor executes the "Under-the-hood" logic, such as evaluating scores, providing adaptive feedback, and managing the state-driven unlocking of curriculum nodes.
- Admin (Administrative Actor): A specialized user responsible for system maintenance and auditing. The Admin utilizes the "Bypass" protocol to oversee all 24 nodes without the constraints of linear progression.

5.1.3 List of Use Cases

- UC-01: User Registration and Authentication.
- UC-02: Initial Diagnostic Assessment and Placement.
- UC-03: Adaptive Interest Profiling.
- UC-04: Dashboard Interaction and Map Navigation.
- UC-05: Learning Module Content Consumpt
- UC-06: Dynamic Exam Execution and Scoring.

- UC-07: Progress Persistence and Node Unlocking.
- UC-08: Administrative Progression Bypass.

5.1.4 Diagram Rationale and Include Relationships

As illustrated in Figure 5.1 , the system utilizes "Include" relationships to represent mandatory automated processes:

1. << include >> Unlock Subsequent Nodes: Every time a student successfully completes an exam (UC-04), the System / AI is triggered to update the database and grant access to the next node.
2. << include >> Provide Personalized Feedback: During the onboarding phase (UC-02), the system analyzes the diagnostic results to offer smart, adaptive feedback that sets the student's starting proficiency level.

5.2 Detailed Use Case Descriptions

UC-01: User Registration and Authentication

Attribute	Description
Use Case ID	UC-01
Actor	Student
Precondition	The user has a valid email address and access to the platform URL.
Main Flow	1. The user navigates to the registration page. 2. The user enters their Name, Email, and Password. 3. The system calls the Appwrite account.create API to initialize the user profile. 4. Upon success, the user is redirected to the login screen to enter credentials. 5. The system establishes a secure session via Appwrite Auth.
Postcondition	The student is authenticated, and a unique User ID is generated and stored in the session.
Alt. Flow	If the email is already registered, the system displays an error message prompting the user to log in instead.

UC-02: Initial Diagnostic Assessment and Placement

Attribute	Description
Use Case ID	UC-02
Actor	Student
Precondition	User has logged in for the first time; proficiency_level is null in the database.
Main Flow	1. The system detects a "New User" status and redirects the actor to the Diagnostic Exam page. 2. The student completes a series of questions across Math, Science, and English. 3. The system calculates the score percentage. 4. Based on thresholds (e.g., <50%, 50-80%, >80%), the system assigns "Foundation," "Beginner," or "Intermediate." 5. The proficiency level is saved to the Appwrite profiles collection.
Postcondition	The student is redirected to the main Dashboard to begin Node 1.

UC-06: Dynamic Exam Execution and Scoring

Attribute	Description
Use Case ID	UC-06
Actor	Student
Precondition	The student has completed the Learning Module and clicked "Start Exam."
Main Flow	1. The system identifies the current nodeID and the student's chosen_interest (Computer, Sport, or Art). 2. The Exam Engine retrieves questions tagged with that specific interest context. 3. The student selects answers for all questions. 4. Upon clicking "Submit," the system evaluates answers and calculates the total score and time elapsed. 5. A Results Modal appears, displaying total EXP earned.
Postcondition	If the score is a "Pass," the system triggers the unlocking of the subsequent node.

UC-08 : Administrative Progression Bypass

Attribute	Description
Use Case ID	UC-08

Actor	Admin
Precondition	The user must be logged in with a specifically authorized Administrative User ID.
Main Flow	1. The Dashboard component initializes and executes a check on the userID attribute. 2. The system identifies the ID as an Administrator (e.g., userId === 'student1'). 3. The state-driven isLocked variable is overridden to false for all 24 nodes. 4. All padlock icons are removed, and all node links become active.
Postcondition	The Admin can freely enter any learning module or exam for auditing without completing prerequisites.